

COMP90038 Algorithms and Complexity

Priority Queues, Heaps and Heapsort

Junhao Gan

Week 7 Lecture 13

Semester 1, 2026

The Priority Queue

Priority Queue is an *abstract data type*. Specifically, a priority queue supports the following operations on a “pool” of elements, each of which is associated with a *priority*:

- *getTop()*: return the item with the *largest priority*
- *pop()*: return and remove the item with the *largest priority*
- *push(x)*: push a new item x with associated priority
- *isEmpty()*: return whether a priority queue is empty

Other operations may be relevant, for example:

- *update(x, p)*: update the priority of item x to p
- *construct()*: construct a priority queue from a list of items
- *join(Q₁, Q₂)*: join the two given priority queues Q_1 and Q_2 into one

Two Special Instances of Priority Queues

Indeed, Stacks and Queues can be considered as two special instances of priority queue.

Think: Can you think of how?

If we treat the *time stamp* of an item when it is pushed as its priority:

- for Stack, an item pushed at a “later” time stamp has larger priority.
- for Queue, an item pushed at an “earlier” time stamp has larger priority.

Possible Implementations of the Priority Queue

Assume **priority = key** and n is the total number of items.

- Unsorted Array:
 - $push(x)$: $O(1)$
 - $pop()$: $O(n)$
- Sorted Linked List:
 - $push(x)$: $O(n)$ (and $O(\log n)$ if a binary search tree is maintained)
 - $pop()$: $O(1)$
- In this lecture, we introduce a data structure called **Binary Heap** which can achieve:
 - $push(x)$: $O(\log n)$
 - $pop()$: $O(\log n)$

Heaps and Priority Queues

Heap is a useful data structure for implementing priority queues.

There are various types of Heap, e.g., **Binary Heap**, **Fibonacci Heap**, **Quake Heap**, and etc.

Due to the practicality and simplicity, in this subject, we mainly focus on the **Binary Heap**.

Unless state otherwise, **heap** refers to **the binary heap** by default.

Maximum (Binary) Heap

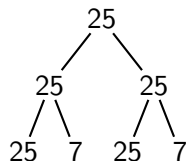
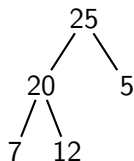
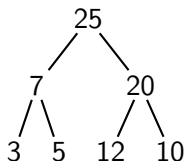
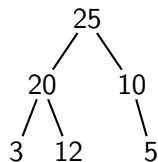
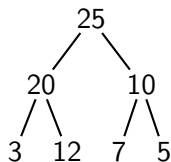
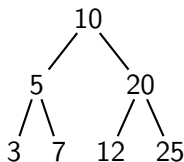
A **maximum (binary) heap** (**max-heap** for short) on *a multi-set of key values*, i.e., *priorities*, is a *rooted binary tree* satisfying the followings:

- each key must occur in *at least one* node in the tree; and
- **Heap Condition:** for any node v in the tree, the *key* of v is *no less than* the key of each of its children.

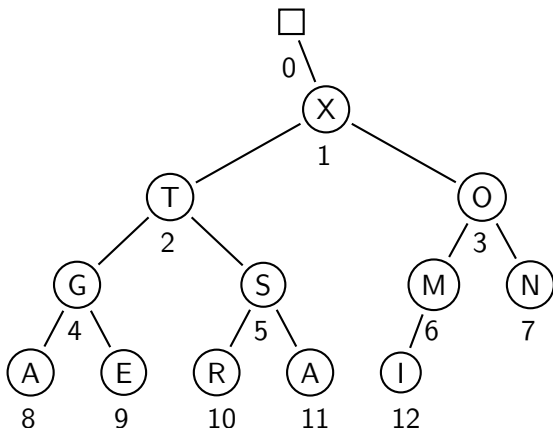
By definition, the *root* of any *sub-tree* of a max-heap has the *largest* key among all the keys of the nodes in the sub-tree.

Heaps and Non-Heaps

Which of these are max-heaps?



A Complete Binary Tree Implementation of Heaps



We can utilise the completeness of the tree and place its elements in level-order in an **array** H .

Note that the children of node at index i will be nodes at indexes $2i$ and $2i + 1$ if they exist.

H :

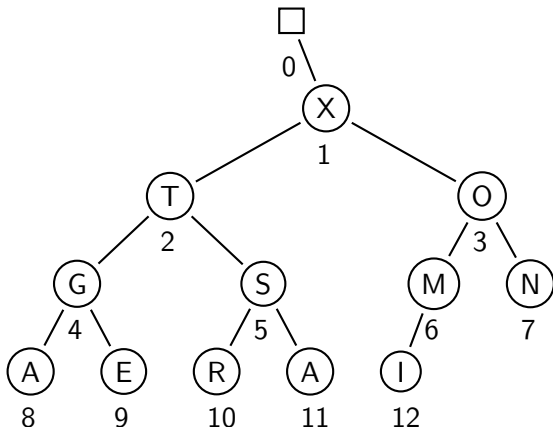
	X	T	O	G	S	M	N	A	E	R	A	I
0	1	2	3	4	5	6	7	8	9	10	11	12

A Complete Binary Tree Implementation of Heaps

In this way, the heap condition is very simple:

For all $i \in \{2, \dots, n\}$, we must have $H[i] \leq H[i/2]$ (integer division).

And the height of the heap is bounded by $O(\log n)$.



H :

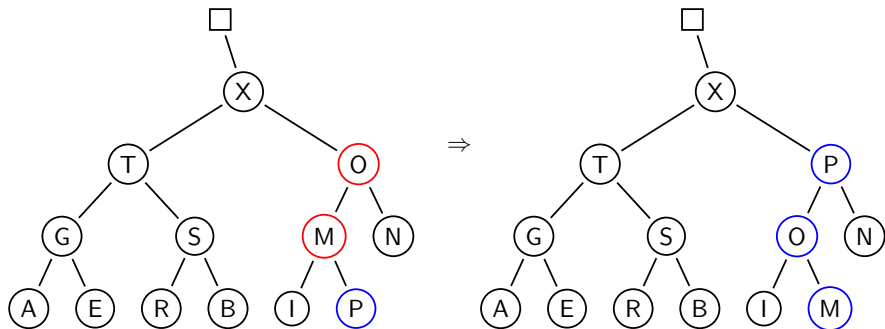
	X	T	O	G	S	M	N	A	E	R	A	I
0	1	2	3	4	5	6	7	8	9	10	11	12

Heap Property (Condition) Maintenance

We first implement two useful operations: *bubble-up*(u) and *bubble-down*(u) which respectively move the element $H[u]$ **up (towards the root)** and **down (towards a leaf)** to a proper location to maintain the heap property.

Bubble-Up

bubble-up(13):



Heap Property (Condition) Maintenance

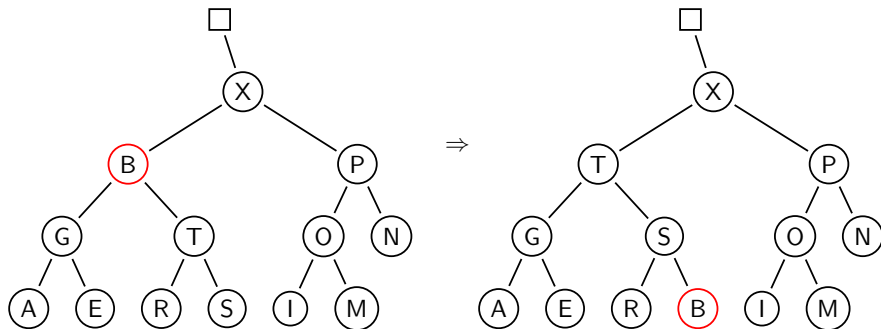
bubble-up(u):

- (a) Set $i \leftarrow u$.
- (b) If $i = 1$, stop.
- (c) If $H[i] \leq H[\lfloor i/2 \rfloor]$, then stop.
- (d) Otherwise, swap $H[i]$ and $H[\lfloor i/2 \rfloor]$ and assign $i \leftarrow \lfloor i/2 \rfloor$.
- (e) Repeat from step (b).

Question: What is the worst-case time complexity of bubble-up? $O(\log n)$

Bubble-Down

bubble-down(2):



Heap Property (Condition) Maintenance

bubble-down(u):

- (a) Set $i \leftarrow u$ and $j \leftarrow \text{null}$.
- (b) If $i = n$, stop.
- (c) If the left child of $H[i]$ exists (indexed at $\ell \leftarrow 2i$), then set $j \leftarrow \ell$.
- (d) If the right child of $H[i]$ exists (indexed at $r \leftarrow 2i + 1$) and $H[r] > H[\ell]$, then $j \leftarrow r$.
- (e) If $j = \text{null}$ or $H[i] \geq H[j]$, stop.
- (f) Otherwise, swap $H[i]$ and $H[j]$; set $i \leftarrow j$ and $j \leftarrow \text{null}$; and repeat from step (b).

Question: What is the worst-case time complexity of bubble-down? $O(\log n)$

Detailed Implementations

- $push(x)$:
 - 1 Insert x into $H[n + 1]$ and let $u \leftarrow n + 1$ and set $n \leftarrow n + 1$.
 - 2 Invoke $bubble-up(u)$.
- $pop()$:
 - 1 If $n = 1$, remove $H[1]$ and stop.
 - 2 Otherwise, i.e., $n > 1$, swap $H[1]$ with the last element $H[n]$.
 - 3 Remove $H[n]$ from the array and set $n \leftarrow n - 1$.
 - 4 Let $u \leftarrow 1$ and invoke $bubble-down(u)$.
- $update-key(u, p)$:
 - 1 If $p = H[u]$, then stop.
 - 2 If $p < H[u]$, set $H[u] \leftarrow p$ and invoke $bubble-down(u)$.
 - 3 If $p > H[u]$, set $H[u] \leftarrow p$ and invoke $bubble-up(u)$.

Question: What is the worst-case time complexity of each of the above operations? $O(\log n)$

Constructing a Max-Heap

Question: Given a set P of n priorities, how would you construct a Max-Heap on P ?

A straightforward solution: push each priority *one by one* into an initially empty max-heap.

Question: What is the worst-case time complexity then? $O(n \cdot \log n)$

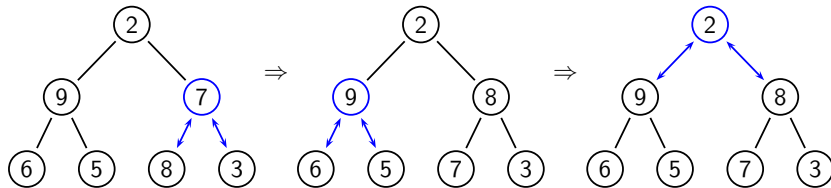
Constructing a Max-Heap: Heapify

Indeed, there exists a well-known algorithm, called **Heapify**, to construct max-heaps on a given set P of n priorities in $O(n)$ time.

The basic idea is to start the construction from the *second last level* in a bottom-up level-by-level manner.

Specifically, the **Heapify** algorithm is as simple as just one line of code:

- for $i = n/2, \dots, 1$, invoke *bubble-down*(i);



Constructing a Max-Heap: Heapify

Analysis:

At first glance, as the worst-case cost of *bubble-down* is $O(\log n)$, and there are $n/2$ *bubble-down*'s invoked.

The worst-case time complexity of Heapify *looks* still $O(n \cdot \log n)$ time.

Interestingly, if we have a closer look, the worst-case cost of *bubble-down* at a node at level ℓ (counting from the leaf level which is at level 0) is actually bounded by $O(\ell)$.

Therefore, by a more careful calculation, the worst-case cost of Heapify can be calculated as:

$$\sum_{\ell=1}^h \sum_{\text{nodes at level } \ell} \ell \leq \sum_{\ell=1}^h \ell \cdot \frac{n}{2^\ell} \leq 2 \cdot n \in O(n),$$

where the root is at level $h = \lfloor \log_2 n \rfloor$. The formal proof of the above calculation is given in the exercises.

Heapsort

Given an unsorted array $H[1] \dots H[n]$, **Heapsort** works as follows:

Step 1 Heapify H into a heap.

Step 2 Invoke $pop()$ operation for $n - 1$ times.

The worst-case time complexity of Heapsort is bounded by $O(n \cdot \log n)$.

Properties of Heapsort

On average slower than quicksort, but stronger performance guarantee.

Truly in-place.

Not stable.